

Smart Door Unlock Security System using Arduino Uno and ESP32 cam module

Team members

VK Ashwin Karthik – 21BEC1454

J Alan Jones – 21BEC1476

S Gowtham Mallaya – 21BEC1590

Sai Krishna S – 21BEC1804

Faculty Name: **Prof Gowri Prasood**

Course Name: **Embedded C**

Course Code: **BECE320E**

Slot: **G1**

Abstract

This project introduces a cutting-edge IoT-driven solution for security in systems. By leveraging the capabilities of Internet of Things (IoT) technology, the system integrates multiple layers of authentication, including RFID, numeric keypad, and remote access control. Central to its architecture are the ESP32 microcontroller and Arduino Uno board, which synergize to orchestrate seamless functionality.

The core strength of this system lies in its multi-factor authentication approach, which fortifies traditional door locks with advanced layers of security. Automated locking and unlocking mechanisms streamline user experience while maintaining stringent security protocols. Additionally, the system incorporates intrusion detection and real-time monitoring features, enabling prompt responses to potential security breaches.

Security is paramount in IoT implementations, and this project addresses this concern by implementing robust encryption protocols and establishing secure communication channels. Fail-safe mechanisms further enhance reliability, ensuring uninterrupted operation even in the face of unexpected events.

One of the standout features of this system is its modular design, which facilitates effortless integration with emerging technologies. Future enhancements such as integration with smart home systems, biometric authentication, and cloud-based access management are readily accommodated, ensuring scalability and adaptability to evolving security needs.

Ultimately, this project serves as a compelling demonstration of how IoT and embedded systems can revolutionize access control, offering both heightened security and unparalleled convenience for residential and commercial environments. By embracing innovation, it sets a precedent for the future of secure access management.

KEYWORDS—IoT (Internet of Things), Multi-Factor Authentication, Door Lock System, ESP32, RFID (Radio Frequency Identification), Keypad, Arduino, Integration, Security, Access Control, Embedded Systems, Home Automation, Smart Home, Wireless Connectivity, Remote Access, Microcontroller, LCD Display

Index

Chapter	Title	Pg no.
0	Abstract	2
1	Introduction	4
	Platform/Technology	5
2	Literature Survey	7
	Problem Definition	9
3	Flowchart	10
	Schematic Diagram	11
	Module wise Explanation	12
4	Results	19
	Code	22
	References	32

Chapter 1

Introduction

In today's digital landscape, safeguarding homes and workplaces has never been more critical. Traditional lock systems often fall short in thwarting unauthorized access attempts. Enter the era of advanced technology integration into access control systems, offering a compelling solution to fortify security while enhancing convenience. This project sets out to tackle these challenges head-on by crafting an innovative IoT-enabled multi-factor authentication door lock system. Harnessing the ESP32 microcontroller, RFID technology, numeric keypad, Arduino Uno board, and LCD Display, it promises a robust defense against intrusions.

At its core, this system combines three potent authentication factors: RFID cards/fobs, numeric passcodes, and IoT connectivity for remote access control. By demanding multiple authentication forms, it drastically slashes the vulnerability to unauthorized entry, delivering a heightened level of security suitable for residential and commercial settings alike.

Fueling its operations is the ESP32 microcontroller, acting as the nerve center orchestrating seamless integration with an RFID reader and keypad module. Users can secure entry by presenting a valid RFID card/fob, punching in a correct passcode, or a combination of both, thereby amplifying security through multi-factor authentication.

Augmenting its capabilities is the Arduino Uno board, facilitating interface with an array of actuators and sensors including electromagnetic door locks, motion detectors, and alarms. This augmentation empowers the system with automated locking/unlocking, intrusion detection, and real-time monitoring functionalities, amplifying both security and convenience.

With IoT connectivity enabled by the ESP32's Bluetooth prowess, the system extends its reach beyond physical confines. Users gain remote access and control over the door lock system, enabling monitoring, access permission management, real-time alerts, and even temporary access provisioning through an intuitive mobile or web interface. This seamless integration ensures heightened security and control, accessible from anywhere, anytime.

Platform/Technology

Arduino IDE



ESP32 Cloud for face registration



Chapter 2

Literature Survey

Patil [1] et al. focuses on smart door lock systems, which are designed to enhance security and convenience compared to traditional mechanical locks. The proposed system uses face recognition, fingerprint, RF card, password, and IoT technologies to unlock the door. The system includes a camera sensor for face detection, a fingerprint sensor for fingerprint recognition, and an RF card reader for contactless access. The security system is also designed with the elderly in mind, as it is difficult for them to open the door manually. The importance of smart door lock systems is emphasized, as they offer automation and convenience in unlocking door locks without the need for mechanical contact. The drawbacks of traditional lock systems, such as the need for mechanical keys and the time delay in unlocking, are highlighted. The use of RFID technology for unlocking doors is also discussed, as it eliminates the need for physical keys and provides a more secure and convenient way to access doors. The paper also covers the use of smartphone-based unlocking systems, such as the Android-based unlocking smart door lock system, which uses a smartphone with an Arduino application for remote control. The use of Bluetooth and GSM modules for remote control and monitoring is also discussed. In addition, the literature review covers the use of various sensors and modules in smart door lock systems, such as the ZigBee module for wireless communication and the solenoid lock for electronic locking. The importance of solenoid locks in smart door lock systems is emphasized, as they operate under electromagnetic force and are easy to operate.

Shanthini[2] et al. proposed system is an IoT-enhanced smart door locking system that uses Bluetooth technology and an Android application to control the door lock. Users can provide login credentials like a username and password, which are verified in a database over the internet. If the credentials are invalid, the system sends an alert via SMS and a buzzer alarm. The system can be scaled to commercial applications like ATMs and vending machines by using other wireless communication. The system's hardware and software requirements include an Arduino UNO, servo motor, piezo buzzer, Bluetooth module (HC-05), and a firebase database to store and retrieve user credentials. The system is designed to improve security without complicating accessibility.

Priyakanth[3] et al. The proposed methodology for door access control combines two contemporary technologies: Internet of Things (IoT) and face recognition. This innovative approach begins with the enrollment process, where individuals register their facial features, which are then stored in the memory of an ESP32 CAM, serving as a centralized repository. This step essentially establishes a database of authorized users, facilitating subsequent access control. Once enrolled, the system is ready to detect and verify individuals seeking access. When a person approaches the door, the ESP32 CAM, acting as a sensor, captures their facial image. This image is then processed through the Arduino IDE software, which contains the necessary code to conduct facial recognition. By comparing the captured face with the enrolled faces stored locally on the ESP32 CAM, the system determines whether the individual is authorized to enter. A crucial aspect of this methodology is the reliance on the ESP32 CAM as both a controller and a memory storage unit. This design choice allows for efficient processing and comparison of facial images directly on the device, eliminating the need for external controllers or additional hardware. However, the methodology acknowledges the possibility of using alternative setups, such as employing a Raspberry Pi connected to a web camera, albeit with increased complexity and potentially higher resource requirements. The functionality of the access control system extends beyond facial recognition to include the operation of physical security mechanisms. A relay, linked to the ESP32 CAM, governs the functioning of a solenoid lock, which secures the door. The relay's state, whether ON or OFF, dictates the status of the solenoid lock, thereby controlling access to the premises. This integration of hardware components with facial recognition technology ensures a comprehensive and robust access control solution. Authentication is based on the comparison of facial features, utilizing algorithms to determine the similarity between the captured face and the enrolled faces in the database. If the system detects a match with a high degree of feature similarity, it triggers the relay module, leading to the unlocking of the solenoid lock and granting access to the authorized individual. Conversely, if the captured face does not align with any enrolled faces, the door remains securely locked, preventing unauthorized entry. Furthermore, the system incorporates a proactive security measure by integrating an email alert mechanism. In the event of an intruder attempting unauthorized access, the system

automatically sends an email alert to the designated owner or administrator. This feature enhances security by providing real-time notifications of potential security breaches, allowing for timely intervention and response.

Karthik[4] et al The methodology proposed for building an economical and low-budget biometric lock system using a smartphone's fingerprint sensor presents a comprehensive approach to integrating hardware and software components for secure access control. The process begins with the setup of communication between an Arduino Nano microcontroller and a smartphone via Bluetooth. This communication link is crucial for enabling the microcontroller to receive and execute commands sent by the smartphone, facilitating remote control of the lock mechanism. A key aspect of the methodology lies in the handling of device IDs and fingerprint authentication. The program developed for the Arduino Nano includes the creation of a string variable to store a unique device ID for the lock. Upon receiving data via Bluetooth, the microcontroller reads this device ID and verifies it against the stored value. This serves as an initial layer of security to ensure that only authorized devices can control the lock. Fingerprint authentication is delegated to the smartphone application, which utilizes the device's built-in fingerprint sensor. Once a user's fingerprint is scanned and authenticated within the application, the corresponding device ID is sent to the Arduino Nano via Bluetooth. If the received device ID matches the one stored in the microcontroller, the servo motor connected to the physical lock is activated, moving the lock to the open position. This seamless integration of fingerprint authentication with Bluetooth communication enhances the security of the system while ensuring user convenience. In terms of application development, the methodology offers flexibility by providing options for creating the Android application using either Android Studio. The latter is favored for its ease of use and avoidance of lengthy code writing. Components such as Bluetooth client, fingerprint module, list view, and image button are added to the application layout to facilitate user interaction and device communication. The integration of the hardware components with the physical lock is straightforward, involving the attachment of the servo motor's moving shaft to the lock lever for movement. All electronic components are then housed within the lock framework, ensuring a compact and integrated system. Power is supplied to the lock using a 5-9V battery, providing portability and flexibility in installation. Once the system is assembled and powered, the smartphone's Bluetooth is paired with the Bluetooth module connected to the Arduino board. Upon opening the application, the Bluetooth icon transitions to a lock icon, indicating readiness for lock control. This user-friendly interface enhances the accessibility and usability of the biometric lock system.

Problem statement

In today's fast-paced world, ensuring the safety and convenience of our homes has become more critical than ever. However, traditional lock systems, reliant on keys, are increasingly proving inadequate in safeguarding against modern security threats. Vulnerabilities to unauthorized access persist, posing significant risks to homeowners and their families. Therefore, there arises a pressing need for a sophisticated, multi-layered authentication system that not only fortifies home security but also streamlines access for authorized individuals.

In response to this evolving security landscape, this project seeks to pioneer an innovative solution: the development of an Automatic Face Recognition Smart Door Lock with a Camera. By harnessing cutting-edge technology, such as facial recognition algorithms and integrated camera systems, this smart lock system aims to revolutionize home security. Gone are the days of fumbling for keys or worrying about lost or stolen access credentials. Instead, homeowners will enjoy seamless and secure access, authenticated through their unique facial features.

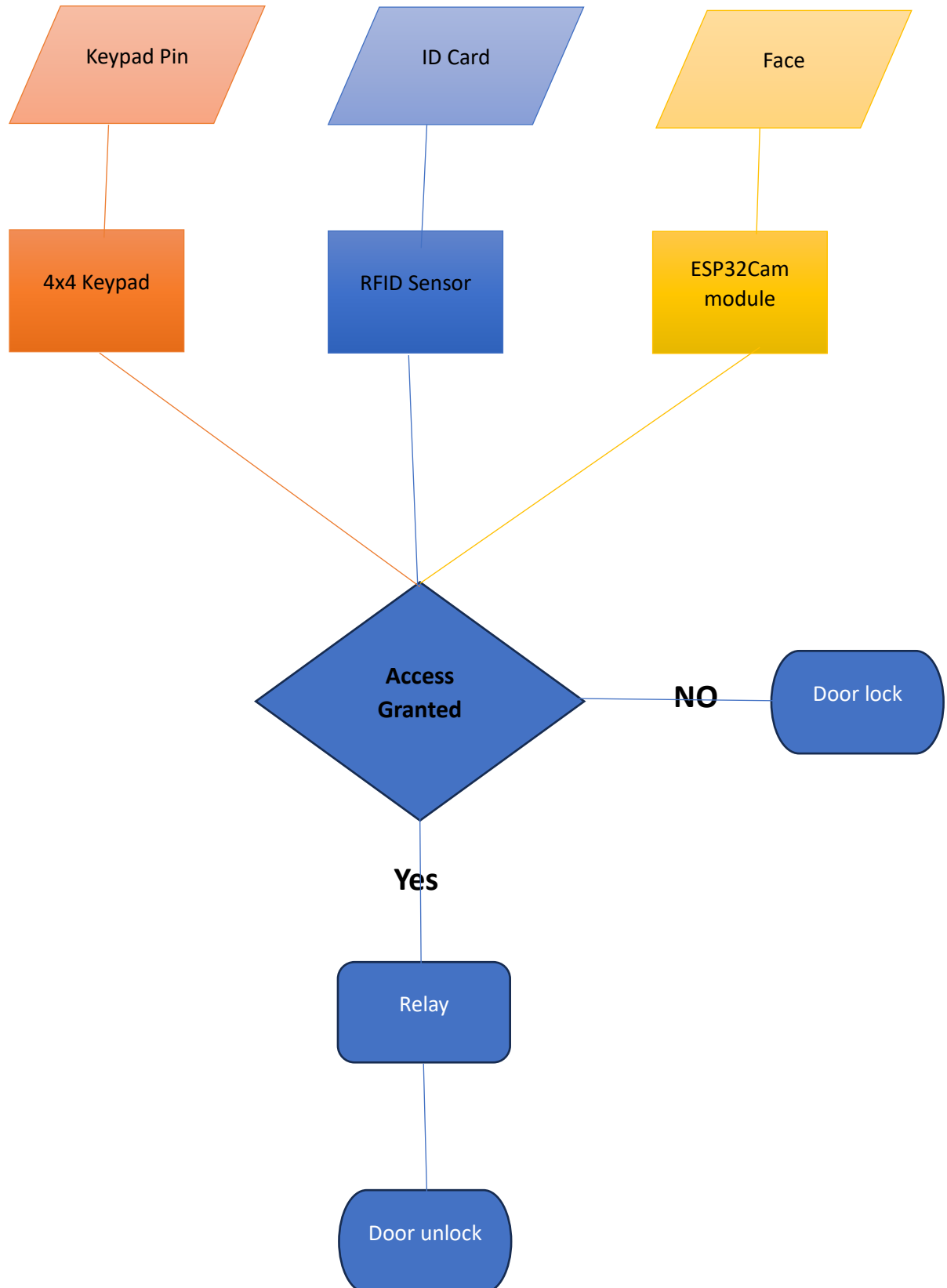
This project recognizes the inherent shortcomings of traditional lock systems and endeavors to bridge the gap with a solution that marries advanced security features with unparalleled convenience. By leveraging facial recognition technology, the proposed system offers a robust defense against unauthorized entry, effectively mitigating the risks associated with key-based vulnerabilities.

Furthermore, the integration of a camera component adds an additional layer of security, enabling real-time monitoring and recording of access events. This not only serves as a deterrent to potential intruders but also provides homeowners with invaluable peace of mind, knowing that their property is under constant surveillance.

In short, this project addresses the pressing need for a modernized approach to home security. By combining advanced authentication methods with user-friendly features, it strives to redefine the paradigm of access control, setting a new standard for intelligent and secure home protection in the digital age.

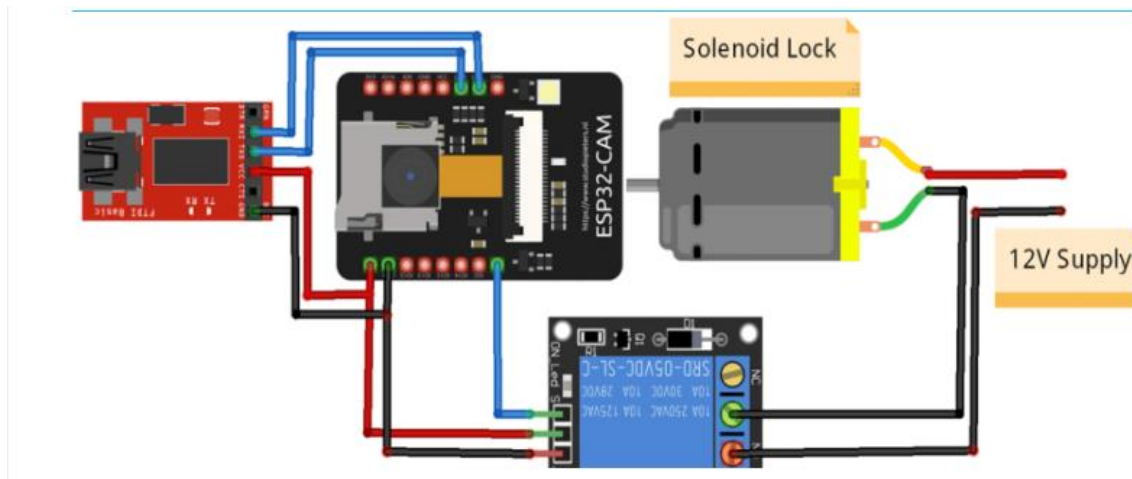
Chapter 3

Flowchart

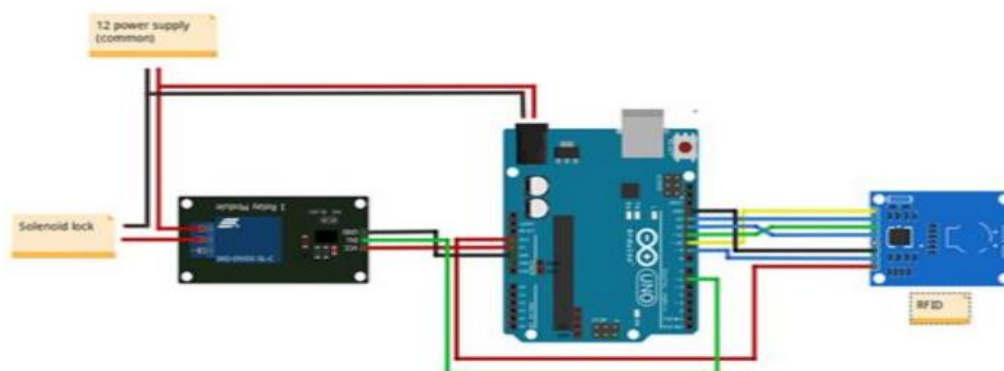


Schematic Diagram

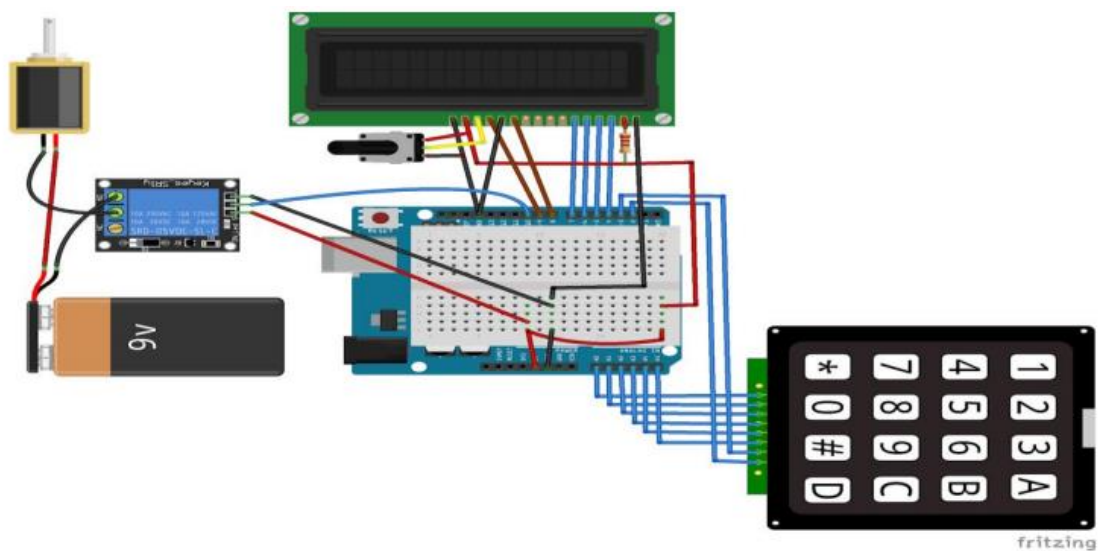
We unlocked the door lock using the ESP32 Camera. We utilized an FTDI232 USB to Serial interface device to program the ESP32CAM. The ESP32CAM and FTDI232 are now linked. We need to connect GPIO 0 to the ESP32CAM's GND pin in order to upload the code. We will use the details we supplied in our code to connect to our hotspot after uploading it. We will look for IP addresses on the serial monitor after connecting. We launch the esp32 Cloud app on Google Chrome, Turn ON the Face Detection and Face Recognition and click on the start stream button. Click on the Enroll face and start taking the samples, when everything is done successfully the electronic lock will open and also the white LED will turn ON. You can see the text welcome on the recognized face.



The MFRC522 IC is the foundation of the RC522 RFID module. We used the Arduino IDE to write the program. We will get our tag information in hex numbers using the Arduino IDE, which we will use to put in the code. We will execute the code by connecting one pin to the relay module. There is a solenoid lock attached to the relay. Access is denied if it is not detected; if it is, the door is unlocked and we will see Access authorized on the LCD.



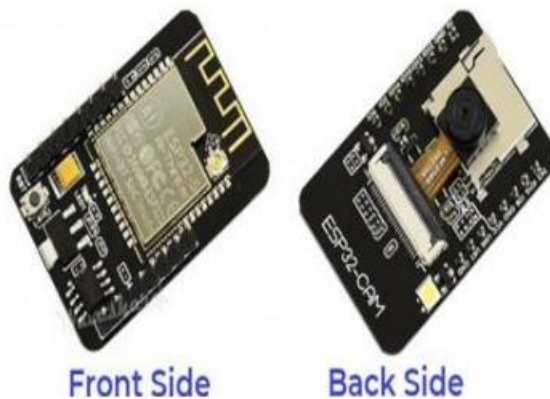
Using the Arduino IDE, we created a program to open the door lock using a 4 * 4 keypad. We'll run the code after entering the password. The relay module will have one pin connected to it. There is a solenoid lock attached to the relay. The LCD will display "Access granted" if the password is recognized, "Access denied" if it is not



Module wise explanation

ESP32 Cam

The ESP32-CAM is a compact, low-power camera module based on the ESP32 microcontroller. It features the OV2640 2MP camera sensor and an onboard microSD card slot. With a tiny footprint of 40x27mm and deep sleep current as low as 6mA, it is well-suited for intelligent IoT applications requiring wireless video monitoring, image uploads, QR code scanning, face recognition, and more.



This module adopts a DIP package design, allowing easy integration into various IoT hardware products. It offers high-reliability wireless connectivity with support for Wi-Fi, classic Bluetooth, and BLE Beacon. Powered by dual 32-bit LX6 CPUs with 7-stage pipeline architecture, it can operate at frequencies between 80MHz to 240MHz. Additionally, the ESP32-CAM integrates several onchip sensors, including a Hall sensor and temperature sensor, making it suitable for smart home devices, industrial wireless control, wireless monitoring systems, and other IoT applications requiring compact, low-power, and versatile camera capabilities.

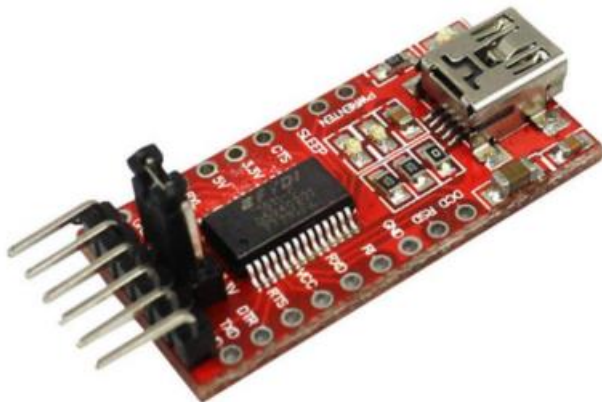
Arduino Uno

Arduino is an open-source electronic device that provides developers and developers with free, flexible and easy to use hardware and software. The platform allows you to build different types of single board microcomputers that the developer community can provide for different types of uses. Microcontrollers are integrated circuits that can write instructions you type using the programming language available in the Arduino IDE environment. These instructions allow you to create programs that interact with the board's circuits. The Arduino microcontroller has communication ports and input/output ports. We can use it to connect different types of peripherals to the card. Data from the peripheral devices you connect will be sent to the microcontroller, which will be responsible for processing the data through them.



FTDI Serial interface

It stands for Future Technology Devices International Limited. Serial Communication: FTDI chips enable the conversion between USB and serial protocols, such as RS-232, RS-485, or TTL-level UART. This makes it easier to connect and communicate with devices that use serial interfaces, including microcontrollers, sensors, GPS modules, industrial equipment, and more. The module provides integration of Hi-Speed USB capability (480Mbit/s) with an FPGA, to enable ultra-fast software-based FPGA configuration/reconfiguration and communication using the FTDI royalty free drivers.



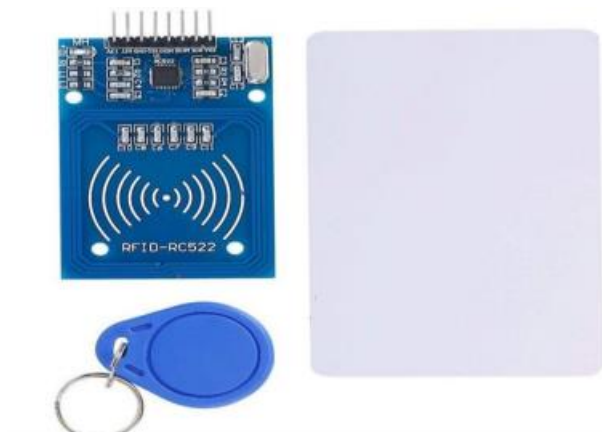
Keypad

4x4 keyboard is a device frequently used in electronics and microcontroller-based projects. It features a thin, flexible membrane with printed materials that form a matrix of buttons in a 4x4 matrix keyboard configuration. A 4x4 matrix keyboard is a loading or input device often used to provide input values to projects. Note that the 4x4 matrix membrane keyboard has 16 equal keys and allows 16 input values. The 4x4 keyboard is unique in that it only uses 8 GPIO microcontroller pins. As we mentioned before, if you need to connect 16 switches, you will need 17 input pins for them to work properly. There are eight pins in the 4x4 keyboard module, as each row and column is connected to a pin. There is also a membrane switch under each key. Arduino knows which key you pressed by looking for lines and lines connected to that key



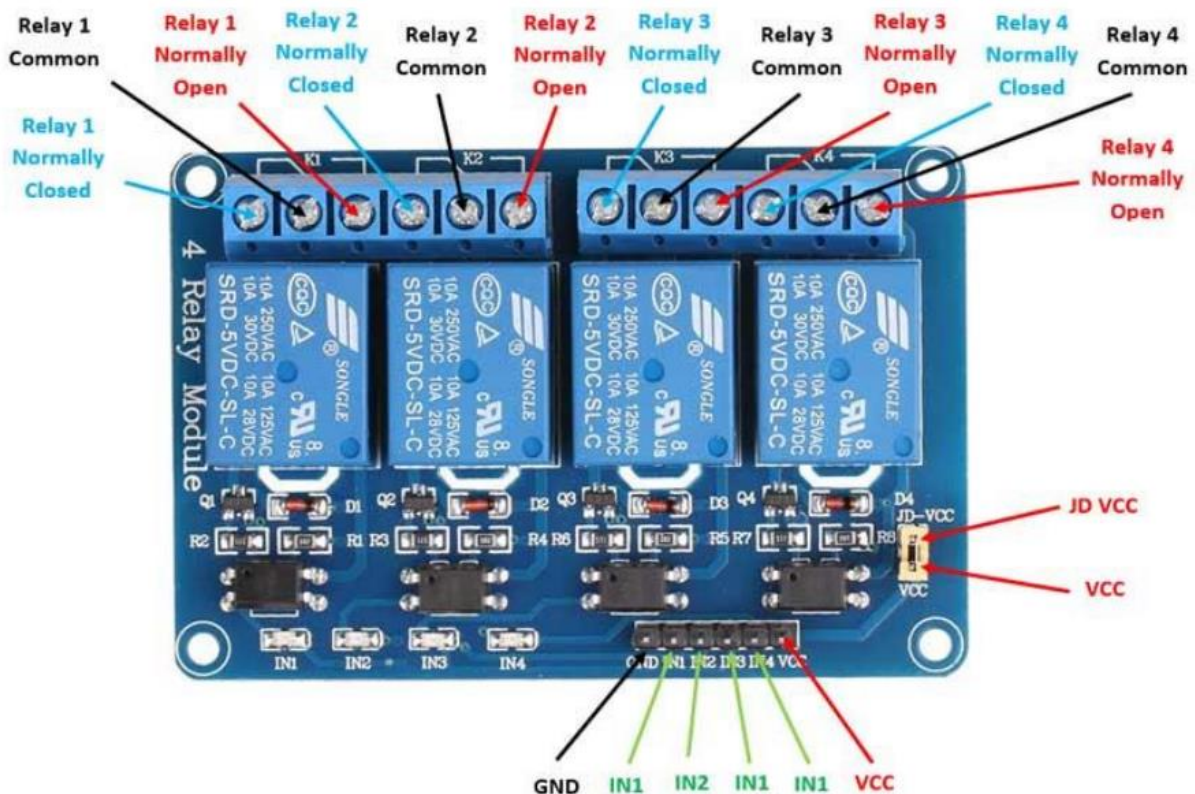
RFID

Radio frequency identification (RFID) refers to a wireless system with two components: a tag and a reader. Radio frequency identification (RFID) is a technology that uses radio waves to passively identify tagged items. It is used in a variety of business and commercial activities, from tracking items in the supply chain to tracking items controlled by libraries. signal label. Radio tags do not transmit their identity and other information to nearby readers and can be disabled or deactivated. Passive RFID tags are powered by a reader and do not require batteries. Active RFID tags are battery operated. RFID tags can store a lot of information, from barcodes to multiple pages of information. The reader can be a handheld mobile phone or can be mounted on a pole or overhead. Lines of text can also be placed on cabinets, rooms or building designs



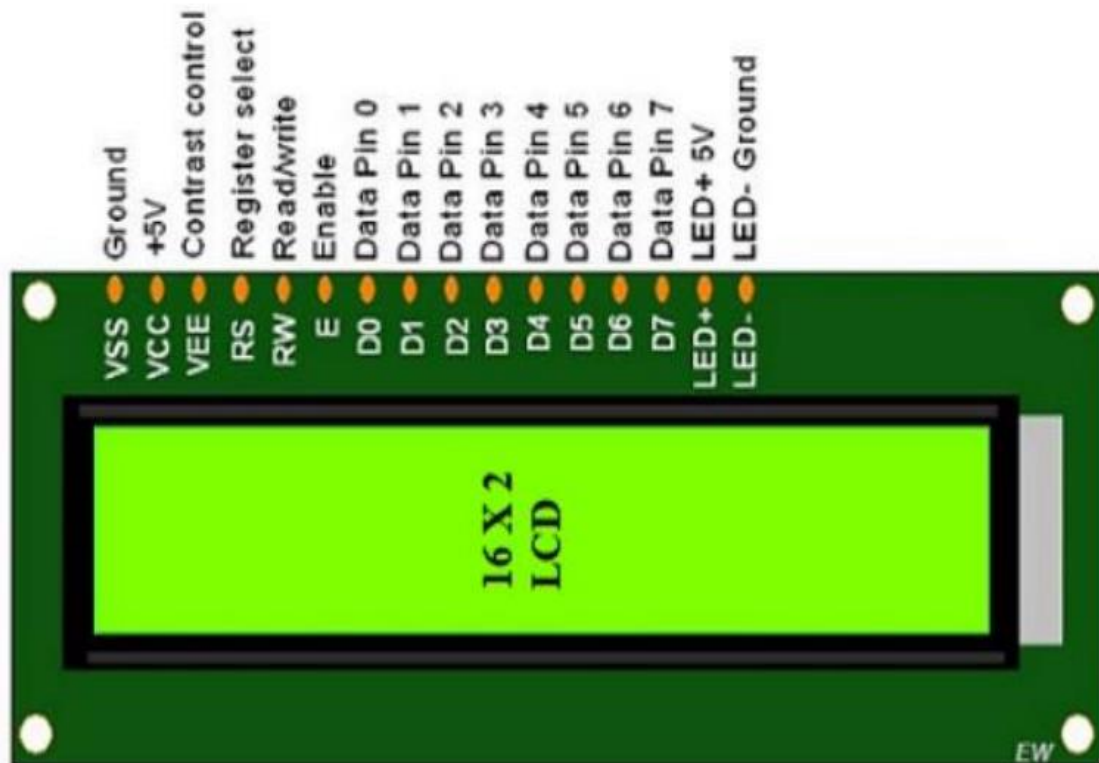
Channel Relay

2.6 4 Channel relay 12v 4 channel relay module interface board allows you to control various electronic devices and other high current devices. It can be controlled directly with microcontroller (Arduino, Raspberry Pi, 8051, AVR, PIC, DSP, ARM, MSP430, TTL logic). The 4channel relay module is a device that allows you to control four independent circuits using a single control signal. It usually has four relays, each of which can switch high voltage such as a lamp, motor, or appliance. This module is designed with input pins that connect to a microcontroller or other control circuit. When a digital signal is sent to one of these input pins, it activates the supply, thus connecting or disconnecting the corresponding circuit. This feature makes the 4channel relay module versatile in automation, IoT and home control where multiple devices need to be controlled or monitored remotely



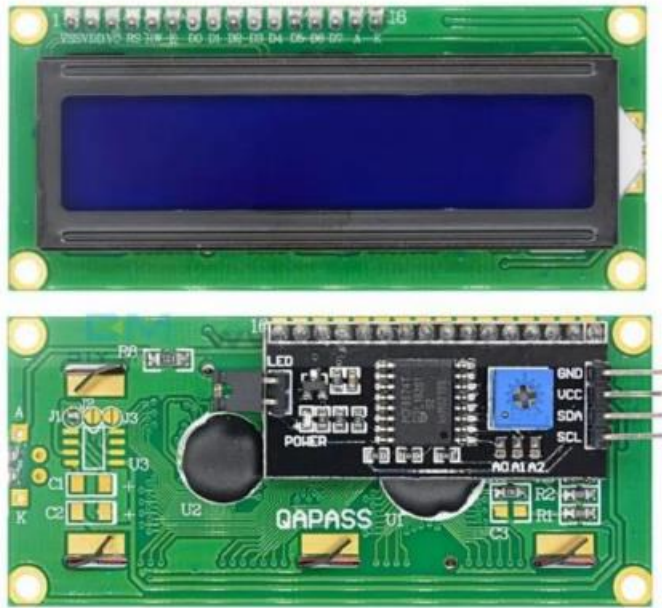
LCD

LCD stands for liquid crystal display. It is an electronic device widely used in many circuits and devices such as mobile phones, computers and televisions. The main benefit of using this mode is its low cost; easy programmability, animation and unlimited display of special symbols, special features and even animations. The screen has 16 lines and 2 lines of alphanumeric characters, allowing 32 characters to be displayed simultaneously. The position of each character on the screen is controlled by a series of pins or signals from the microcontroller. The microcontroller sends commands and data to the screen indicating which characters should be displayed in which positions. The display then interprets these commands and generates the electrical signals necessary to activate the liquid crystal; The liquid crystal selectively blocks or allows light to pass through to create the signals displayed on the screen. These instructions are frequently used in many electronic devices and systems to provide visual feedback, status information, or user intervention



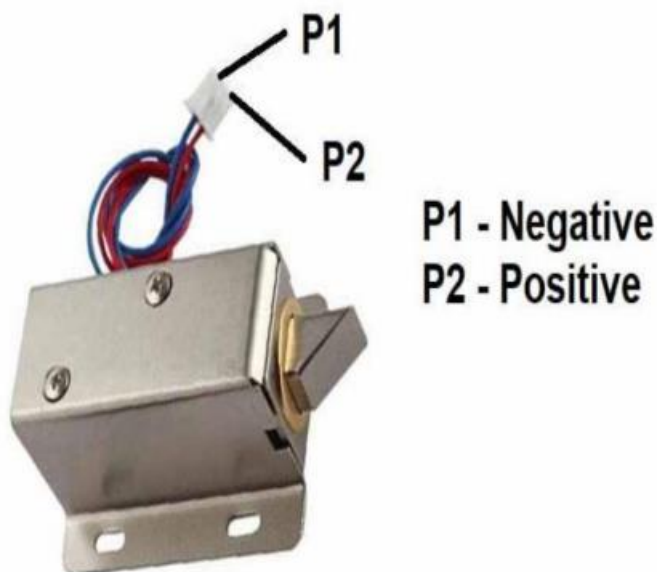
LCD with I2C Interface

Inter Integrated Circuit (I2C for short) is a two-way communication protocol. The 16x2 LCD display with I2C interface combines a standard 16x2 character LCD with the I2C communication protocol. It utilizes only two pins (SDA and SCL) for data transfer, reducing pin usage compared to parallel interfaces. The I2C protocol allows the LCD to receive commands, text data, and control signals like cursor positioning, brightness adjustment, and contrast control over the I2C bus. This design provides a compact, space-saving solution for displaying information in embedded systems and IoT applications where pin counts are limited or ease of integration is crucial. The I2C interface simplifies wiring and communication, making it an efficient and effective way to incorporate text displays into projects.



Solenoid Lock

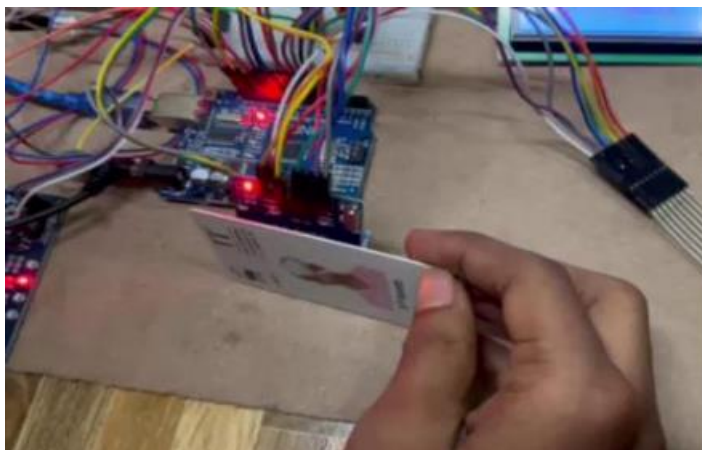
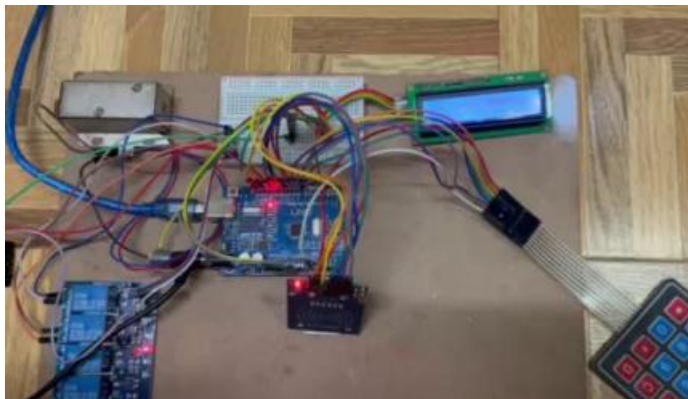
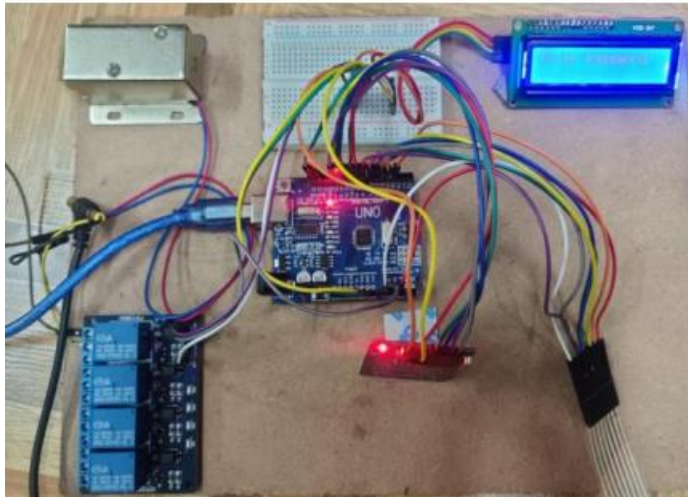
Solenoid locks are electromagnetic locks. Electromagnetic locks refer to electronic locks and latches used for locks. There is a power-on open mode and a boot lock mode that can be selected depending on the situation. Power on mode is activated only when the solenoid valve is energized. Since this type of door is closed, it cannot be opened in case of bad electricity or connection, making it very safe. This type is used only where there is a need to fight terrorism. The energized model locks the door when the solenoid valve is energized. If the power goes out, the door will open. Hold mode performs both locking and unlocking functions by applying a positive or negative pulse to the solenoid and holds the inactive state in each position. This mode saves energy because the solenoid valve does not need to be energized all the time. Continuous measurement is designed to deliver the measured voltage for several hours without the temperature exceeding the nominal value, while irregular measurement is designed to deliver the measured voltage only for the specified period of time without exceeding the temperature rise.



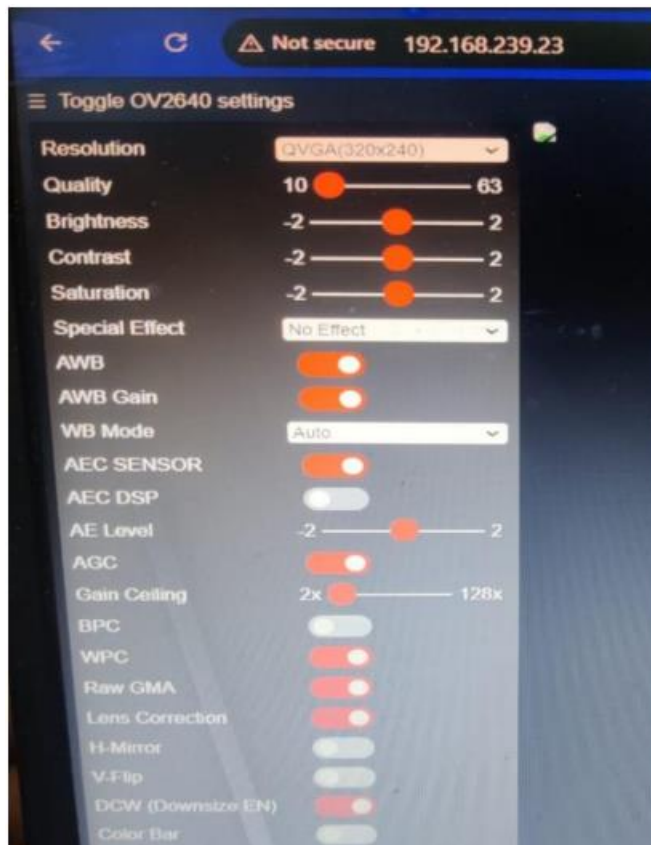
Chapter 4

Results

The outcomes of this work are shown in the figure below. Using an Arduino, RFID, keypad, and an ESP32- CAM, we have developed a smart door lock system. In this variant, in order to unlock the door, a user must first enter the password. Alternatively, they can use the card to scan the RFID barcode, or the ESP32-CAM can recognize their face to unlock the door.







Code

Keypad and RFID code

```
#include <Keypad.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <SPI.h>

#include <MFRC522.h>


#define SS_PIN 10

#define RST_PIN 5


int done = 0;


MFRC522 rfid(SS_PIN, RST_PIN);


byte keyTagUID[][4] = {{0x15, 0xB2, 0x7A, 0x32},{0xF5, 0x1D, 0xD3,
0x5C},{0xC5, 0x7D, 0x61, 0x5B}}; // Array to store authorized RFID tag UIDs

String name[] = {"Akshith","Ashwin Karthik","Sandeep Krishna"}; // Array to
store corresponding names


// Set the LCD address to 0x27 for a 16 chars and 2 line display
LiquidCrystal_I2C lcd(0x27, 16, 2);


const byte ROWS = 4; // Number of rows on the keypad
const byte COLS = 4; // Number of columns on the keypad


// Define the keypad layout
```

```
char keys[ROWS][COLS] = {
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'}
};

int i=0;
int j=0;
int reset = 2;

byte colPins[COLS]= {6, 8, 7, 14};
byte rowPins[ROWS] = {16,3,4,5};
// Create the Keypad object
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// Define the expected password
const char passcode[] = "1235";
char enteredCode[sizeof(passcode)] = ""; // To store entered passcode
int relay = 15;

void setup() {
    Serial.begin(9600);
    pinMode(relay,OUTPUT);
    pinMode(reset,OUTPUT);
    digitalWrite(relay,HIGH);
```

```
lcd.init(); // Initialize LCD
SPI.begin(); // Initialize SPI bus
rfid.PCD_Init(); // Initialize MFRC522

// Turn on the backlight
lcd.backlight();
lcd.clear();
lcd.setCursor(i,j);
lcd.print("Enter password");
}

void loop() {
    // Check for keypad input
    char key = keypad.getKey();
    if (key != NO_KEY) { // Check if a key is pressed
        Serial.println(key);
        if (strlen(enteredCode) < sizeof(passcode) - 2) { // Check if entered code is
not complete yet
            Serial.println(key);
            strcat(enteredCode, &key); // Append the pressed key to the entered code
            lcd.setCursor(0,1);
            lcd.print(enteredCode); // Print the entered code so far
        } else { // Entered code is complete
            strcat(enteredCode, &key); // Append the pressed key to the entered code
            Serial.println(enteredCode);
            if (strcmp(enteredCode, passcode) == 0) { // Compare entered code with
the expected passcode
```

```

    lcd.clear();
    i=0;
    j=0;
    lcd.setCursor(i,j);
    lcd.print("Access Granted!"); // If the entered code matches, print Access
Access Granted
    digitalWrite(relay,LOW);
    delay(5000);
    digitalWrite(relay,HIGH);
    lcd.clear();
    lcd.setCursor(i,j);
    lcd.print("Enter password");
    i++;
} else {
    i=0;j=0;
    lcd.clear();
    lcd.setCursor(i,j);
    lcd.print("Access Denied!"); // If the entered code doesn't match, print
Access Denied
    delay(2000);
    lcd.clear();
    lcd.setCursor(i,j);
    lcd.print("Enter password");
    i++;
}
memset(enteredCode, 0, sizeof(enteredCode)); // Clear the entered code
}

```

```
}
```

```
// Check for RFID tag
```

```
if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
```

```
    for(int i=0;i<sizeof(keyTagUID)/sizeof(keyTagUID[0]);i++){
```

```
        if (rfid.uid.uidByte[0] == keyTagUID[i][0] &&
```

```
            rfid.uid.uidByte[1] == keyTagUID[i][1] &&
```

```
            rfid.uid.uidByte[2] == keyTagUID[i][2] &&
```

```
            rfid.uid.uidByte[3] == keyTagUID[i][3] ) {
```

```
            Serial.print("Access is granted - ");
```

```
            Serial.println(name[i]);
```

```
            lcd.clear();
```

```
            lcd.setCursor(0,0);
```

```
            lcd.print("Access Granted!");
```

```
            delay(5000);
```

```
            lcd.clear();
```

```
            digitalWrite(relay,LOW);
```

```
            delay(5000);
```

```
            digitalWrite(relay,HIGH);
```

```
            delay(1000);
```

```
            lcd.setCursor(0,0);
```

```
            lcd.print("Enter password");
```

```
            done=1;
```

```
        }
```

```
    }
```

```
    if (done==0){
```

```
        i=0;j=0;
```

```
        lcd.clear();
```

```
        lcd.setCursor(i,j);
```

```
        lcd.print("Access Denied!"); // If the entered code doesn't match, print
```

```
Access Denied
```

```
        delay(2000);
```

```
        lcd.clear();
```

```
        lcd.setCursor(i,j);
```

```
        lcd.print("Enter password");
```

```

        i++;
    }
    rfid.PICC_HaltA();
    rfid.PCD_StopCrypto1();

}
done=0;
}

```

Face detection in ESP32

```

#include "esp_camera.h" // Include the ESP32 camera library
#include <WiFi.h> // Include the ESP32 WiFi library

// Define camera model (uncomment the desired model)
//#define CAMERA_MODEL_WROVER_KIT
//#define CAMERA_MODEL_ESP_EYE
//#define CAMERA_MODEL_M5STACK_PSRAM
//#define CAMERA_MODEL_M5STACK_WIDE
#define CAMERA_MODEL_AI_THINKER // Selected camera model
#define Relay 2 // Define pin for relay
#define Red 13 // Define pin for red LED
#define Green 12 // Define pin for green LED
#include "camera_pins.h" // Include the camera pin definitions

const char* ssid = "-----"; //Wifi Name SSID
const char* password = "-----"; //WIFI Password

void startCameraServer(); // Function prototype

```

```
boolean matchFace = false; // Initialize variable to track face match status
boolean activateRelay = false; // Initialize variable to track relay activation status
long prevMillis=0; // Initialize variable to store previous millis
int interval = 5000; // Define interval for relay activation (in milliseconds)

void setup() {
    pinMode(Relay,OUTPUT); // Set relay pin as output
    pinMode(Red,OUTPUT); // Set red LED pin as output
    pinMode(Green,OUTPUT); // Set green LED pin as output
    digitalWrite(Relay,LOW); // Set relay pin to low
    digitalWrite(Red,HIGH); // Set red LED pin to high (initially off)
    digitalWrite(Green,LOW); // Set green LED pin to low (initially off)
    Serial.begin(115200); // Initialize serial communication
    Serial.setDebugOutput(true); // Enable debug output
    Serial.println(); // Print empty line

    camera_config_t config; // Declare camera configuration struct

    // Define camera pin configuration
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
```

```
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
```

```
// Initialize camera with specified configurations
```

```
if(psramFound()){
```

```
    config.frame_size = FRAMESIZE_UXGA; // Use UXGA frame size if PSRAM is
found
```

```
    config.jpeg_quality = 10; // Set JPEG quality
```

```
    config.fb_count = 2; // Set framebuffer count
```

```
} else {
```

```
    config.frame_size = FRAMESIZE_SVGA; // Use SVGA frame size if PSRAM is not
found
```

```
    config.jpeg_quality = 12; // Set JPEG quality
```

```
    config.fb_count = 1; // Set framebuffer count
```

```
}
```

```
#if defined(CAMERA_MODEL_ESP_EYE)

    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif

// Initialize camera
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err); // Print error message
    if camera initialization fails
        return;
}

sensor_t * s = esp_camera_sensor_get(); // Get camera sensor information

// Adjust sensor settings if the camera model is OV3660
if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1); // Flip sensor vertically
    s->set_brightness(s, 1); // Increase brightness
    s->set_saturation(s, -2); // Decrease saturation
}

s->set_framesize(s, FRAMESIZE_QVGA); // Set frame size to QVGA (320x240)

#if defined(CAMERA_MODEL_M5STACK_WIDE)
    s->set_vflip(s, 1); // Flip sensor vertically
```

```
s->set_hmirror(s, 1); // Mirror sensor horizontally
#endif

WiFi.begin(ssid, password); // Connect to WiFi network

while (WiFi.status() != WL_CONNECTED) { // Wait until WiFi connection is
established
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected"); // Print WiFi connection status

startCameraServer(); // Start camera server

Serial.print("Camera Ready! Use 'http://"); // Print message indicating camera
is ready
Serial.print(WiFi.localIP());
Serial.println("' to connect");
}

void loop() {
    if(matchFace==true && activateRelay==false) // If face match is detected and
relay is not activated
    {
        activateRelay=true; // Set relay activation flag to true
        digitalWrite(Relay,HIGH); // Activate relay
    }
}
```

```

digitalWrite(Green,HIGH); // Turn on green LED
digitalWrite(Red,LOW); // Turn off red LED
prevMillis=millis(); // Record current millis
}

if (activateRelay == true && millis()-prevMillis > interval) // If relay is activated
and interval has passed
{
    activateRelay=false; // Reset relay activation flag
    matchFace=false; // Reset face match flag
    digitalWrite(Relay,LOW); // Deactivate relay
    digitalWrite(Green,LOW); // Turn off green LED
    digitalWrite(Red,HIGH); // Turn on red LED
}
}

```

References

1. Venkatasamy, R., Dhanraj, J. A., Aravinth, S., & Balachandar, K. (2022, August). Design and Development of IOT based Smart Door Lock System. In *2022 Third International Conference on Intelligent Computing Instrumentation and Control Technologies (ICICICT)* (pp. 1525-1528). IEEE.
2. Shanthini, M., Vidya, G., & Arun, R. (2020, August). IoT enhanced smart door locking system. In *2020 Third International Conference on Smart Systems and Inventive Technology (ICSSIT)* (pp. 92-96). IEEE.
3. Patil, K. A., Vittalkar, N., Hiremath, P., & Murthy, M. A. (2020). Smart door locking system using IoT. *International Research Journal on EngTechnol (IRJET)*, 3090-3094.
4. Priyakanth, R., Krishna, N. S., Karanam, G., Prassna, M. L., Poojitha, S. B., & Mounika, J. (2021, October). IoT based smart door unlock and intruder alert system. In *2021 2nd International Conference on Smart Electronics and Communication (ICOSEC)* (pp. 6-11). IEEE.
5. circuitdigest.com/microcontroller-projects/esp32-cam-face-recognition-door-lock-system